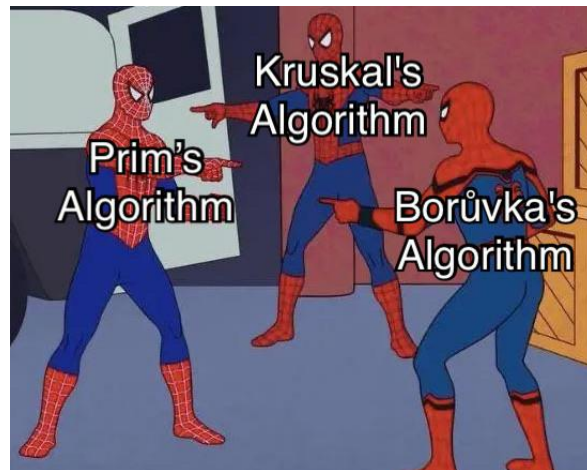


Chapitre 6

Recherche d'arbres couvrants de poids minimal



On travaille dans $G = (S, A, p)$ un graphe non-orienté pondéré. On cherche un arbre de la forme $\alpha = (S, A')$, $A' \subset A$ de poids minimal, qu'on appellerait « **arbre couvrant de poids minimal** ».

Définition - Arbre

Un arbre est un graphe $G = (S, A)$

1. acyclique et connexe
2. connexe tq si on enlève une arête il n'est plus connexe
3. connexe avec n sommets et $n - 1$ arêtes
4. acyclique avec n sommets et $n - 1$ arêtes
5. $\forall x, y \in S^2, \exists!$ chemin entre x et y
6. libre / acyclique et générateur/connexe

et ces définitions sont équivalentes

DÉMONSTRATION.

- (1) $\Rightarrow$ (2) : Un graphe acyclique et connexe possède la caractéristique décrite en (2). En effet, si on lui enlevait une arête et qu'il conservait sa connexité, on observerait un cycle. Ce qui est absurde.
- (2) $\Rightarrow$ (3) : Soit n le nombre de sommets de G . On sait que G est connexe mais ne l'est plus si on lui enlève une arête. Cette propriété est valable pour les graphes tq. $|A| = n - 1$ (sinon on trouve des contre-exemples bien choisis pour les graphes à plus de n arêtes). Cette propriété est valable pour tous ces graphes car tout graphe possédant $n - 2$ arêtes a au minimum un sommet isolé, ce qui ferait perdre la connexité du graphe de notre hypothèse.
- (3) $\Rightarrow$ (4) : Un graphe connexe avec n sommets et $n - 1$ arêtes doit nécessairement être acyclique car si il possédait un cycle, on aurait un sommet isolé (car seulement $n - 1$ arêtes), ce qui rendrait impossible le fait qu'il soit connexe.
- (4) $\Rightarrow$ (5) : Supposons (par l'absurde) qu'il existe deux chemins différents entre deux sommets $x, y \in S$, alors on pourrait parcourir le deuxième chemin dans l'autre sens et donc générer un cycle, ce qui contredit notre hypothèse. D'où l'absurdité.
- (5) $\Rightarrow$ (1) : Si il existe un unique chemin pour chaque paire de sommets, ce graphe est bien connexe. Enfin, comme pour l'implication précédente, l'unicité du chemin nous garantit le caractère acyclique.
- (6) $\Leftrightarrow$ (1) : par définition.

Proposition

Un graphe connexe a au moins $|S| - 1$ arêtes.

DÉMONSTRATION.

Montrons ce résultat par récurrence sur le nombre de sommets :

- $n = 1$: Ce graphe a donc au moins 0 arêtes.
- $H_1, \dots, H_n \Rightarrow H_{n+1}$: Soit G un graphe connexe à $n + 1$ sommets. Soit $u \in S$ et soit G_u le graphe induit de G sur $S \setminus \{u\}$. Soient $C_1, \dots, C_t$ les composantes connexes de G_u , alors en notant n_i le nombre de sommets de C_i et m_i son nombre d'arêtes, on a par H.R que

$$\forall i \in \llbracket 1; t \rrbracket, m_i \geq n_i - 1$$

En sommant, on a

$$\sum_i m_i \geq \sum_i n_i - 1 = n - t$$

Et $\sum_i m_i = \text{nb d'arêtes de } G_u = |A| - \deg(u)$, or puisque G était connexe, $\deg(u) \geq t$, d'où

$$|A| \geq n = (n + 1) - 1$$

H_{n+1} vraie. Ce qui clôt la récurrence.

Proposition

Un graphe acyclique a au plus $|S| - 1$ arêtes.

DÉMONSTRATION.

Montrons ce résultat par récurrence sur le nombre de sommets :

□ $n = 1$: immédiat

□ $H_1, \dots, H_n \Rightarrow H_{n+1}$: Soit $G = (S, A)$ à $n+1$ sommets et m arêtes et soit $u \in S$. Notons G_u le graphe induit de G et reprenons les mêmes notations qu'à la preuve précédente. On a alors par H.R :

$$\forall i \in \llbracket 1; t \rrbracket, m_i \leq n_i - 1$$

Donc, en sommant :

$$|A| - \deg(u) = \sum_{i=1}^t m_i \leq n - t$$

donc

$$\deg(u) \geq t + (|A| - n)$$

Or, comme G est acyclique, alors $\deg(u) \leq t$ car sinon on aurait une composante connexe à laquelle u serait relié 2 fois et on aurait alors un cycle (ce qui est absurde).

Ainsi,

$$|A| \leq n = (n+1) - 1$$

H_{n+1} vraie. Ce qui clôt la récurrence.

1 Position du problème

Soit $G = (S, A, p)$ **connexe** et on cherche $\alpha = (S, A')$, $A' \subset A$ qui minimise $\sum_{(x,y) \in A'} p(x, y)$

EXISTENCE

Proposition

Tout graphe connexe admet un arbre couvrant.

DÉMONSTRATION.

Soient $G = (S, A)$ un graphe connexe et $\mathcal{P} = \{ G' = (S, A') \mid A' \subset A \text{ et } G' \text{ connexe} \}$.

Notons $\mathcal{P}_{\text{card}} = \{ |A'| \mid (S, A') \in \mathcal{P} \}$. Ainsi,

- $\mathcal{P}_{\text{card}} \subset \mathbb{N}$
- $\mathcal{P}_{\text{card}} \neq \emptyset$ car $G \in \mathcal{P}$

Donc $\mathcal{P}_{\text{card}}$ admet un plus petit élément $n_0 = |A_0|$ avec $G_0 = (S, A_0) \in \mathcal{P}$.

Finalement, G_0 est connexe et (par construction : minimalité de n_0), si on enlevait une arête à G_0 , on obtiendrait un graphe non-connexe. D'où le fait que $G = (S, A_0)$ soit un arbre couvrant.

UNICITÉ

Théorème

Si un graphe connexe pondéré a tous ses poids différents alors il admet un unique arbre couvrant de poids minimal.

DÉMONSTRATION. Raisonnons par contraposée.

Soit $G = (S, A, p)$ admettant 2 arbres couvrants de poids minimal. Notons-les $\begin{cases} \alpha_1 = (S, A_1) \\ \alpha_2 = (S, A_2) \end{cases}$ avec $A_1 \neq A_2$.

$\Rightarrow$ Montrons que G admet 2 arêtes de poids identiques, de poids minimal dans chacun de ces ensembles.

Soient $e_1 \in A_1 \setminus A_2$ et $e_2 \in A_2 \setminus A_1$ de poids minimal dans ces deux ensembles (qui existent bien car $A_1 \neq A_2$, et $|A_1| = |A_2| = |S| - 1$) et supposons sans perte de qualité que $p(e_1) \leq p(e_2)$.

Considérons $G' = (S, A_2 \cup \{e_1\})$.

Ainsi, G' n'est plus acyclique car $|(A_2 \cup \{e_1\})| = |S| > |S| - 1$.

Soit C un tel cycle, alors nécessairement $e_1 \in C$ (sinon α_2 ne pourrait pas être un arbre) et il existe $e_3 \in A_2 \setminus A_1$ tq. $e_3 \in C$ (sinon α_1 ne pourrait pas être un arbre). On peut avoir $e_3 = e_2$ mais pas nécessairement, dans tous les cas on a bien la chaîne d'inégalités suivante :

$$p(e_3) \geq p(e_2) \geq p(e_1) \Rightarrow \boxed{p(e_3) \geq p(e_1)}$$

Soit $G'' = (S, (A_2 \setminus \{e_3\}) \cup \{e_1\})$, ainsi $p(G'') = p(\alpha_2) - p(e_3) + p(e_1) \leq p(\alpha_2)$.

G'' est un arbre couvrant car possède $|S| - 1$ arêtes et G'' est connexe car α_2 l'est donc les chemins de α_2 n'empruntant pas e_3 sont conservés et tout chemin empruntant e_3 donne lieu à un chemin dans G'' (car on a un cycle avec des arêtes de A_2 et qu'on a bien rajouté e_1).

Finalement, $p(G'') = p(\alpha_2)$ par minimalité de α_2 .

Donc $p(e_1) = p(e_3)$, induisant le fait que tous les poids ne sont pas distincts.

D'où le résultat par contraposée.

Néanmoins ceci n'est valide qu'avec les poids différents : **que faire si les poids ne sont pas différents?**

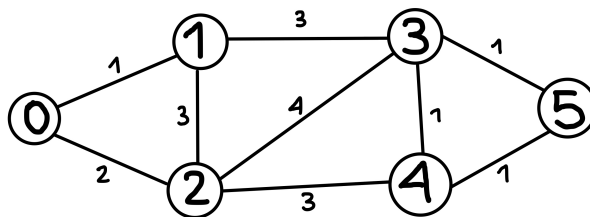
On peut toujours donner des critères pour décider quelle arête choisir quand on veut considérer celle de poids minimal même en cas d'égalité.

Critère de différenciation

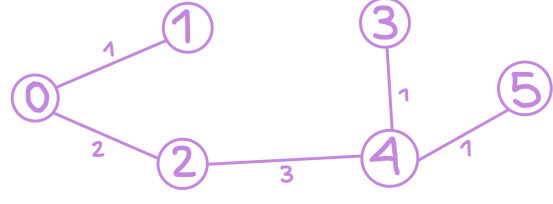
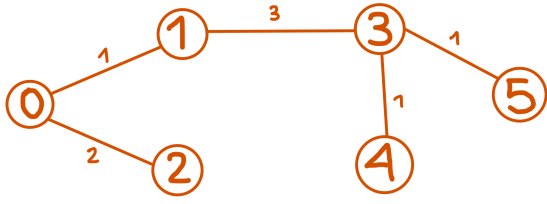
Soient $(i, j), (i', j') \in A$.

- si $p(i, j) > p(i', j') \rightarrow (i', j')$
- si $p(i, j) < p(i', j') \rightarrow (i, j)$
- sinon :
 - si $\min(i, j) < \min(i', j') \rightarrow i, j$
 - si $\min(i, j) > \min(i', j') \rightarrow i', j'$
 - sinon :
 - si $\max(i, j) < \max(i', j') \rightarrow (i, j)$
 - si $\max(i, j) > \max(i', j') \rightarrow (i', j')$
 - sinon $\rightarrow (i, j) = (i', j')$

EXEMPLE 1.1



Si l'algorithme considère l'arête de poids minimal, il va alors choisir $(0, 1)$. On peut donc considérer pour tout ensemble $A_0 \subset A$ l'unique arête de A_0 et pour H graphe connexe l'unique arbre de poids minimal.



Ces deux arbres ont alors le même poids. Néanmoins si on veut les départager selon le critère précédent : l'arbre orange sera privilégié.

Remarque

En pratique, on se « fiche » complètement de départager les ex-aequo, l'algorithme prendra n'importe quelle arête de poids minimal, mais cette considération d'unicité permet d'un point de vue théorique de parler de l'arbre couvrant de poids minimal.

2 Principe général

Soient $G = (S, A, p)$, $\alpha = (S, A')$ son arbre couvrant de poids minimal. On sélectionne les arêtes une à une de manière gloutonne.

$A_{selec} \leftarrow \emptyset$

À chaque étape, on choisit $e \in A \setminus A_{selec}$ tq $A_{selec} \cup \{e\} \subset A'$.

On garantit donc l'invariant :

$$A_{selec} \subset A'$$

Si je garantis cela, à la fin (i.e après $|S| - 1$ arêtes sélectionnées) on aura $A_{selec} \subset A \Rightarrow \text{Card}(A_{selec}) = \text{Card}(A') \Rightarrow A_{selec} = A'$.

Définition - Arête sûre

Soient $G = (S, A, p)$ et $S' \subset S (S' \neq S)$.

Une arête sûre pour S' dans G est une arête de poids minimal ayant exactement une extrémité dans S'

Théorème

Soient $G = (S, A, p)$ et $S' \subset S (S' \neq S)$, alors toute arête sûre de S' pour G appartient à l'arbre couvrant de poids minimal de G .

DÉMONSTRATION.

Soit $e = (x, y)$ une arête sûre pour S' dans G et soit $\alpha = (S, A')$ l'arbre couvrant de poids minimal de G .

Supposons par l'absurde que $e \notin A'$.

Dans α , il existe un chemin entre x et y qui n'utilise pas e . Notons ce chemin :

$$\underbrace{x}_{\in S'} = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_p = \underbrace{y}_{\notin S'}$$

Notons i_0 le plus petit indice tq. $x_{i_0} \notin S'$ (qui existe bien car $x_p \notin S'$), alors $0 < i_0 \leq p$.

On a alors $(x_{i_0-1}, x_{i_0}) = e'$ une arête avec exactement une extrémité dans S' . On en déduit (sinon il s'agirait d'une arête sûre) :

$$p(e') > p(e)$$

Ainsi, $\alpha' = (S, (A' \setminus \{e'\}) \cup \{e\})$ est un arbre couvrant (car connexe et possède le bon nombre d'arêtes) et

$$\begin{aligned} p(\alpha') &= p(\alpha) - p(e') + p(e) \\ &< p(\alpha) \end{aligned}$$

ce qui est absurde car α est minimal.

Algorithme obtenu

```

 $A' \leftarrow \emptyset$ 
for  $i \leftarrow 1$  to  $|S| - 1$  do
     $(x, y) \leftarrow$  arête sûre pour une des c.c de  $(S, A')$ 
     $A' \leftarrow A' \cup \{(x, y)\}$ 
return  $(S, A')$ 

```

Correction et terminaison de cet algorithme

➤ On a l'invariant, d'après de le théorème précédent,

$$A' \subset A_0$$

où (S, A_0) est l'arbre couvrant de poids minimal de G .

➤ En effet, à chaque étape on augmente $|A'|$ d'exactly un car (x, y) est entre 2 c.c différentes de (S, A') et ne peut donc pas appartenir à A' . Donc on l'invariant au i^e tour de boucle :

$$|A'| = i$$

Ce qui garantit :

- (1) $\forall i \in \llbracket 1; |S| - 1 \rrbracket$, il existe bien au moins 2 composantes connexes et donc au moins une arête sûre.
- (2) Pour $i = |S| - 1$, i.e à la fin de cet algo : $A' \subset A_0$ et $|A'| = |A_0|$. D'où $A' = A_0$.

3 Algo de Kruskal

Principe

- On trie les arêtes par valeur croissante des poids
- Pour chaque arête, dans cet ordre la la sélectionne si elle ne forme pas de cycle

Pour faire cela concrètement, on utilise la structure *union find* qui représente les composantes connexes du graphe induit par les arêtes sélectionnées. On privilégiera une représentation de graphe avec la liste des arêtes. à chaque étape, l'arête sélectionnée est sûre pour une composante connexe du graphe induit par les arêtes sélectionnées précédemment.

Algorithme - Kruskal(g)

```

 $uf \leftarrow$  create(nb de sommets de  $g$ )
 $l \leftarrow$  arêtes de  $g$  triées par poids croissants
for each  $(x, y) \in l$  do
    if find( $uf, x$ )  $\neq$  find( $uf, y$ ) then
        sélectionner  $(x, y)$ 
        union( $uf, x, y$ )

```

Proposition - Complexité de l'algorithme de Kruskal

$$\mathcal{O}(|S| + |A| \log |A|)$$

DÉMONSTRATION.

On a $|A| \propto |S|^2 = o(|A| \log |A|)$, les opérations union find sont négligeables par rapport au tri. On en déduit la complexité recherchée de par la création de la structure, le tri et la boucle ensuite.

4 Algo Prim

Soit donc l'algo de Prim depuis un sommet s

$$A_{\text{selec}} \leftarrow \emptyset \text{ et } S_{\text{couverts}} \leftarrow \{s\}$$

À chaque étape, on sélectionne une arête sûre pour S_{couverts} , on l'ajoute à A_{selec} et on ajoute le sommet à S_{couverts} .
On sélectionne ainsi $|S| - 1$ arêtes.

On remarque S_{couverts} correspond (à partir de la 2ème étape de l'algo) aux sommets des arêtes de A_{selec} , ainsi une arête sûre de S_{couverts} n'appartient pas à A_{selec} .

Proposition - Invariant de l'algorithme de Prim

Si $a = (S, A')$ l'arbre couvrant de poids minimal alors,

$$A_{\text{selec}} \subset A' \wedge \text{Card}(A_{\text{selec}}) \text{ strictement croissant}$$

à chaque tour donc au bout de $|S| - 1$ tours on aura $A_{\text{selec}} = A'$

4.1 Mise en oeuvre

4.1.1 Version 1

On va travailler avec les sommets en utilisant le fait que sélectionner une arête sûre pour S_{couverts} est équivalent à choisir un sommet dans $S \setminus S_{\text{couverts}}$ relié à un sommet de S_{couverts} .

On va utiliser une **file de priorité** contenant les sommets de $S \setminus S_{\text{couverts}}$ associé à la priorité

$$pr(x) = \min_y \{p(x, y) \mid y \in S_{\text{couverts}}, (x, y) \in A\}$$

on ajoute également $\Pi(x) = y, pr(x) = p(x, y)$.

On obtient alors

Algorithme - Prim (version 1)

```

 $\forall x \in S, pr(x) \leftarrow +\infty$ 
 $\forall x \in S, \Pi(x) \leftarrow -1$ 
 $pr(s) \leftarrow 0$ 
 $F \leftarrow S$ 
 $A \leftarrow \emptyset$ 
while  $F \neq \emptyset$  do
   $x \leftarrow \text{defile}(F)$ 
  if  $x \neq s$  then
     $A \leftarrow A \cup \{(x, \Pi(x))\}$ 
  for each  $y$  voisin de  $x$  do
    if  $pr(y) > p(x, y)$  then
       $pr(y) \leftarrow p(x, y)$  et  $F$  est modifiée en conséquence
       $\Pi(y) \leftarrow x$ 

```

Proposition - Complexité de l'algorithme de Prim (version 1)

$$\mathcal{O}(|S| + |S| \log |S| + |A| \log |S|)$$

DÉMONSTRATION.

On a

$$\mathcal{O}(\underbrace{|S|}_{\text{construction tas}} + \underbrace{|S| \log |S|}_{|S| \times \text{défiler}} + \underbrace{|A| \log |S|}_{\text{modif de } F})$$

4.1.2 Version 2

On va mettre $(x, \Pi(x), pr(x))$ dans t

Algorithme - Prim (version 2)

```
 $\forall x, vu(x) \leftarrow \text{false}$   
 $A \leftarrow \emptyset$   
 $F \leftarrow \emptyset$   
insérer  $((s, -1, 0))$  dans  $F$   
while  $F \neq \emptyset$  do  
   $x, \Pi, poids \leftarrow \text{defile}(F)$   
  if  $\neg vu(x)$  then  
     $vu(x) \leftarrow \text{true}$   
    if  $x \neq s$  then  
       $A \leftarrow A \cup \{(x, \Pi)\}$   
    for each voisin  $y$  de  $x$  do  
      enfiler  $(y, x, p(x, y))$  dans  $F$ 
```

Rmq : Ici, $\Pi(x)$ représente le parent de x .

On retrouve une complexité analogue

5 Algorithme de Boruvka (HP)

Algorithme - Boruvka

```
if  $(S, A_{\text{selec}})$  est connexe then  
  return  $(S, A_{\text{selec}})$   
else  
  on calcule ses c.c pour chacune d'entre elle on détermine une arête sure qu'on ajoute à  $A_{\text{selec}}$  et on recommence récursivement.
```